Reporte de práctica: Driver de software para un relevador

Alumno	Jesús Antonio Rodríguez Peraza
Matrícula	231000105
Materia	Sistemas Programables (SCC-1023)
Práctica	10 — Unidad 4, tema 4.1.1 — consigna 4.1.1-A
Plataforma	Arduino UNO R4 WiFi + puente H L298N con motorreductor TT + LED activo en bajo; control por Monitor Serie

Objetivo

Modelar por software un relevador con una clase de C++ que encapsule su estado, sus órdenes y sus reglas de seguridad, y usar esa misma clase para controlar dos actuadores reales con lógica distinta: un canal del L298N con un motorreductor (activo en alto) y un LED conectado como el optoacoplador de un módulo de relevador (activo en bajo).

Metodología

Señal y potencia separadas. Un pin del UNO R4 WiFi entrega unos 8 mA y el motorreductor pide cientos de miliamperes al arrancar, así que el Arduino solo da la orden por D7 a la entrada IN1 del L298N; la energía del motor sale de dos pilas 18650 en serie (≈ 7.4 V) conectadas a la terminal +12V del módulo. Con el jumper de ENA puesto el canal A queda siempre habilitado, e IN2 va a GND: IN1 en ALTO hace girar el motor y en BAJO lo detiene (las dos entradas en BAJO frenan el motor). El L298N funciona así como un relevador electrónico de solo encender y apagar.

La clase `Relevador`**.** Guarda en datos privados el pin, si es activo en bajo, el estado (encendido o apagado), el momento del último cambio medido con `millis()` y el tiempo mínimo entre cambios (1000 ms por defecto en el constructor). Sus métodos públicos son los que pide la consigna: `iniciar()`, `encender()`, `apagar()` y `estaEncendido()`; se agregó `faltaMs()` para que el mensaje de rechazo diga cuánto falta. Los dos objetos se crean de la misma clase: `Relevador motor(7, false)` y `Relevador led(6, true)`.

Traducción de la orden al nivel del pin. El método privado `_nivelPara()` resuelve la lógica en una sola línea: `(encendido != _activoEnBajo) ? HIGH : LOW`. Los `digitalWrite()` viven solo dentro de la clase; el programa principal nunca escribe un pin, solo dice encender o apagar.

Regla del tiempo mínimo. `encender()` y `apagar()` pasan por `_cambiarA()`: si ya pasaron 1000 ms desde el último cambio, actualiza el estado, escribe el pin, guarda `millis()` y regresa `true`; si no, regresa `false` sin tocar el pin. La resta `millis() - _ultimoCambio` con `uint32_t` sigue funcionando cuando `millis()` se desborda (a los ~49 días). Pedir el estado en el que ya está (por ejemplo `M0` con el motor apagado) no conmuta nada: regresa `true` sin tocar el pin ni reiniciar el tiempo, y el Monitor Serie responde igual que si hubiera cambiado.

Estado seguro. `iniciar()` marca el estado como apagado y escribe el nivel de apagado del pin antes y después de `pinMode(OUTPUT)`, para que el actuador no dé un pulso al configurar el pin (en el LED activo en bajo, una salida que arranca en BAJO lo encendería). El arranque cuenta como un cambio: la primera orden se acepta 1 s después de encender o reiniciar el Arduino. En `setup()` los actuadores se inician antes de imprimir el mensaje de bienvenida.

Comandos sin delay(). El `loop()` solo llama a `leerSerie()`, que junta los caracteres del Monitor Serie (9600 baudios, con "Nueva línea") en un búfer de 8 caracteres convertidos a mayúsculas; al llegar el fin de línea, `procesar()` compara la orden con `strcmp()` (así `m1` también sirve). Una orden que no existe responde "Comando desconocido". No hay ningún `delay()` en el programa.

Comando	Acción	Respuesta
M1 / M0	Encender / apagar el motor	"Motor ENCENDIDO" / "Motor APAGADO"
L1 / L0	Encender / apagar el LED	"LED ENCENDIDO" / "LED APAGADO"
E	Estado de ambos	"--- ESTADO ---", "Motor: APAGADO", "LED: ENCENDIDO"
Orden antes de 1 s	Se rechaza, el pin no cambia	"Orden rechazada: espere 640 ms entre cambios"

Diagrama de estados

Se diseñó antes de programar: dos estados, APAGADO como estado inicial (tras `iniciar()` o RESET) y la regla del tiempo mínimo como condición de cada transición.

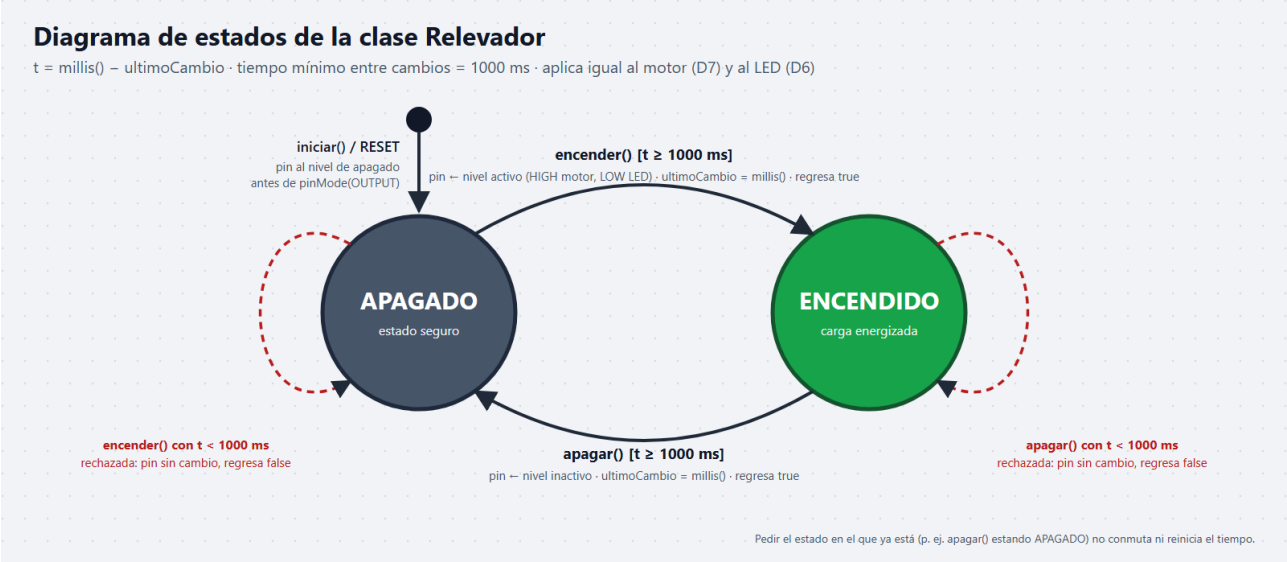


Diagrama de estados de la clase Relevador: las órdenes con t < 1000 ms se rechazan y el pin no cambia.

Conexiones

Parte	Desde	Hacia
Potencia	Positivo del portapilas (2 × 18650)	+12V del L298N
Potencia	Negativo del portapilas	GND del L298N
Potencia	GND del L298N	GND del Arduino (tierra común)
Potencia	OUT1 / OUT2	Motorreductor TT
Señal	D7	IN1 del L298N

Parte	Desde	Hacia
Señal	IN2 del L298N	GND
LED	5V del Arduino	Ánodo del LED (pata larga)
LED	Cátodo del LED	Resistencia de 220 Ω → D6

Los jumpers de ENA y 5V-EN se quedan puestos; la terminal +5V del L298N no se conecta al Arduino y el motor nunca se alimenta desde el 5V del Arduino. Todo se arma con el USB desconectado.

Diagrama de conexión

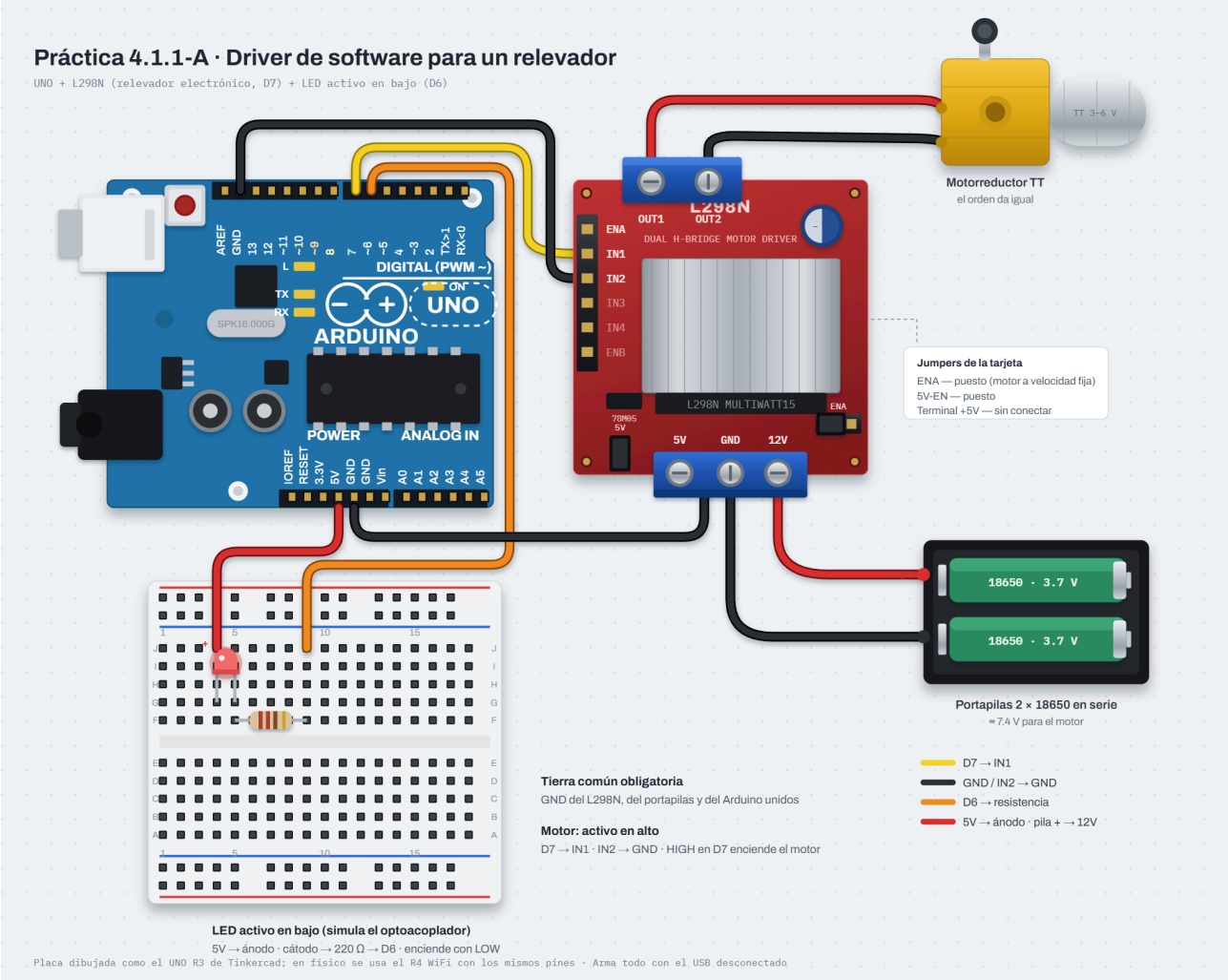
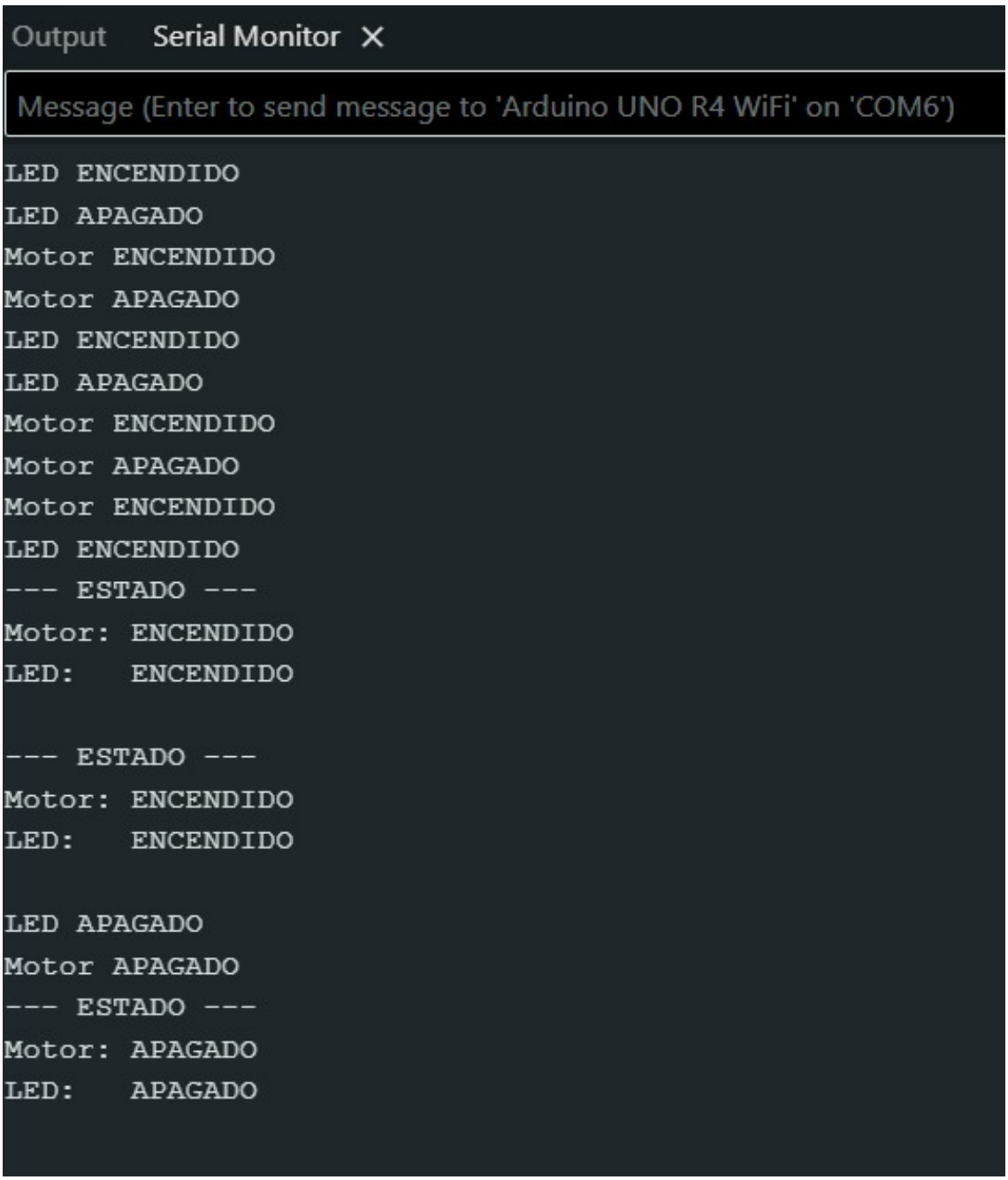


Diagrama de conexión: D7 → IN1 (motor, activo en alto), D6 ← cátodo del LED (activo en bajo), tierra común entre pilas, L298N y Arduino.

Pruebas

Lo observado con el circuito armado:

Prueba	Observado
L1 / L0	El LED enciende y se apaga; el Monitor Serie responde LED ENCENDIDO / LED APAGADO.
M1 / M0	El Monitor Serie responde Motor ENCENDIDO / Motor APAGADO. En el video los cables del motor se sostienen con la mano porque no quedaron fijos en la clema del L298N.
E (estado)	Reporta Motor: ENCENDIDO y LED: ENCENDIDO despues de M1 y L1, y ambos APAGADO despues de L0 y M0: coincide con la ultima orden.
Tiempo minimo 1 s	No se registro: en la captura no aparece ninguna orden rechazada (pendiente repetir M1 y M0 seguidos).



Monitor Serie durante las pruebas (captura del equipo).

Prueba de inversión

Si el objeto del LED se crea por error como activo en alto (`Relevador led(6, false)`), la clase escribe el nivel contrario al correcto en todo momento. Al arrancar, `iniciar()` pone D6 en BAJO pensando que apaga, y el LED se enciende solo aunque el programa diga APAGADO. Con `L1` la clase escribe ALTO y el LED se apaga, con `L0` se vuelve a encender, y `E` reporta siempre lo contrario de lo que se ve. La regla de 1 s sigue funcionando, porque solo depende del tiempo, pero el estado seguro se pierde: con un módulo de relevador real, la carga quedaría energizada al encender el sistema. Por eso la lógica activa en alto o en bajo se declara una sola vez, al crear el objeto, y se revisa contra el circuito.

Preguntas de reflexión

1. ¿Por qué el LED se enciende con LOW y no con HIGH? Explica el recorrido de la corriente.

Porque su ánodo está fijo en 5V y lo que controla el pin es el otro extremo. Con D6 en LOW (0 V) hay una diferencia de 5 V: la corriente sale del pin 5V, entra por el ánodo, sale por el cátodo, pasa por la resistencia de 220 Ω y entra al pin D6, que la lleva a tierra dentro del microcontrolador (el pin *absorbe* la corriente). Con D6 en HIGH los dos extremos quedan en 5 V, no hay diferencia de voltaje y no circula corriente: el LED se apaga. Con un LED rojo (≈ 2 V) la corriente es (5 – 2) / 220 ≈ 14 mA, por encima de los ~8 mA que se recomiendan por pin en la R4; para dejarlo encendido mucho tiempo convendría una resistencia de 470 Ω (≈ 6 mA).

2. ¿Qué ventaja tiene que la regla del tiempo mínimo viva dentro de la clase y no en el loop()?

Que se cumple siempre y no depende de que el programador se acuerde. Cualquier parte del programa que mande encender o apagar pasa por la misma regla, cada objeto lleva su propio `_ultimoCambio` (el motor y el LED no se estorban) y el `loop()` queda limpio: solo interpreta comandos. Si mañana se agrega un tercer relevador o un botón que también lo controle, la protección ya viene incluida; con la regla en el `loop()` habría que copiarla en cada lugar y bastaría un olvido para conmutar demasiado rápido.

3. ¿Por qué un relevador real no sirve para controlar velocidad con PWM?

Porque es un interruptor mecánico: la bobina tarda unos 5 a 15 ms en mover el contacto, y el PWM del Arduino conmuta cientos de veces por segundo (unos 490 Hz). El contacto no alcanza a seguir la señal, se queda vibrando o a medio cerrar, cada conmutación bajo carga produce un arco que quema los contactos y la vida útil, de unos cientos de miles de operaciones, se agotaría en minutos. Para variar velocidad se usa un interruptor electrónico (transistor, MOSFET o el puente H), que conmuta en microsegundos sin desgaste.

4. ¿Por qué el estado seguro es "apagado"? Da un ejemplo industrial donde convenga lo contrario (contacto NC).

Porque cuando el sistema arranca, se reinicia o falla, el controlador no sabe en qué situación está la máquina, y una carga que se energiza sola (un motor, una banda, una prensa) puede lastimar a alguien o dañar el proceso. Apagada, la carga no hace nada inesperado. Lo contrario conviene cuando detener la carga es lo peligroso: el ventilador de extracción de gases de una mina o de un laboratorio químico, o la bomba de enfriamiento de un equipo que se sobrecalienta. Ahí la carga se conecta al contacto normalmente cerrado (NC): si el controlador se cae y la bobina se desenergiza, el contacto NC se cierra y el ventilador o la bomba siguen funcionando.

5. ¿Por qué es obligatoria la tierra común entre el portapilas y el Arduino?

Porque un voltaje siempre se mide entre dos puntos. El L298N decide si IN1 está en ALTO comparándolo con su propia GND, que es el negativo de las pilas. Si esa tierra no está unida a la del Arduino, los 5 V de D7 no tienen una referencia común ni un camino de regreso para la corriente de la señal: la entrada queda flotando y el motor puede no encender, encender solo o responder al azar. Unir las tierras no mezcla las fuentes: el motor sigue alimentándose de las pilas, solo se comparte la referencia de 0 V.

6. Si mañana cambiaras el L298N por un módulo de relevador real, ¿qué partes del programa tendrías que modificar?

Prácticamente solo la línea donde se crea el objeto. La mayoría de los módulos de relevador con optoacoplador son activos en bajo, así que `Relevador motor(7, false)` pasaría a `Relevador motor(7, true)` (y el pin si se conecta en otro). La clase, la regla del tiempo mínimo, el estado seguro y los comandos del Monitor Serie no cambian; lo que cambia es el cableado: VCC, GND e IN del módulo, y la carga en los contactos COM y NO. Esa es la ventaja de tener el modelo de software: el resto del programa no sabe qué hardware hay detrás.

Conclusiones

Encapsular el relevador en una clase hace que el resto del programa hable con órdenes que tienen significado (encender, apagar) y deje de preocuparse por niveles de pin. La misma clase controló un actuador activo en alto y otro activo en bajo cambiando un solo parámetro, y las reglas físicas, el tiempo mínimo entre cambios y el arranque en estado seguro, quedaron dentro del modelo, donde ninguna parte del programa puede saltárselas.

La práctica también deja clara la separación entre señal y potencia: el Arduino solo manda la orden y el L298N, alimentado por las pilas con tierra común, es el que conmuta la carga. Cambiar el L298N por un relevador real se reduce a cambiar el cableado y un parámetro del objeto.

Video

Video del funcionamiento: https://youtube.com/shorts/bL3LOrrr8CQ